ASSESSMENT ITEM 3 | PROJECT REPORT (GROUP)

Real-Time Business Insights: Data Analytics Dashboard Project Plan

Project charter, scope, schedule, resources, risk and issues, change control, communication, quality and configuration management for Harbourline Retail Group

Group members

Student name	Student ID
Gazi Md Tarequr Rahman	68765
Atirikta Pokharel	66660

Project selected: Option 5. Development of a Data Analytics Dashboard for Real-Time Business Insights

Due: Session 11

Note: *Harbourline Retail Group is a hypothetical organisation created for this assessment. All figures are planning estimates.*

Table of Contents

Executive Summary....2
1. Introduction....3
2. Literature Review....4
3. Project Background and Business Case....7
4. Project Charter....9
5. Project Scope Statement....11
6. Work Breakdown Structure....14
7. Project Schedule and Gantt Chart....16
8. Resource Allocation and Responsibility Matrix....20
9. Risk and Issues Management Strategy....22
10. Change Request Management Process....25
11. Communication Management Plan....27
12. Quality Management Approach....29
13. Configuration Management Using GitHub....31
14. Conclusion....33
References....34
Appendix A: Group Contribution....36
Appendix B: Team Collaboration Evidence (GitHub Projects)....37
Appendix C: GitHub Repository Evidence....38

Executive Summary

Harbourline Retail Group operates 40 stores across New South Wales and Victoria and an online store. Managers currently make decisions from weekly Excel reports that are compiled by hand from four separate systems. By the time a report reaches a store manager, the sales and stock data in it are up to seven days old. Stock-outs are found late, promotions cannot be adjusted mid-week, and finance staff spend about 30 hours a week assembling reports.

This report presents the project plan for a **real-time data analytics dashboard** that will stream point-of-sale (POS) and e-commerce transactions into a cloud data platform and show sales, inventory and customer KPIs in Microsoft Power BI, refreshed within five minutes. The project runs for **31 weeks (1 February to 3 September 2027)** with a budget of **AUD $452,760**, including a 10% contingency reserve.

The plan covers all of the elements required by the assessment brief:

- **Literature review (Section 2):** research on IT project failure, project management frameworks, real-time analytics and project control practices, and how it shaped this plan.
- **Project charter and scope statement (Sections 4 and 5):** sponsor, governance and team roles; objectives, deliverables, inclusions and exclusions, acceptance criteria, assumptions and constraints.
- **WBS and Gantt chart with dependencies (Sections 6 and 7):** 23 work packages and 27 activities and milestones linked by finish-to-start dependencies. The critical path runs through requirements, the data platform build, dashboard development, testing and rollout.
- **Resource allocation and RACI matrix (Section 8):** who works on what and when, linked to the budget, with one accountable owner for every deliverable.
- **Risk and issues management strategy (Section 9):** a 5 × 5 likelihood and impact matrix, a nine-item risk register and an issue log with an escalation path.
- **Change request management process (Section 10):** a seven-step process run by a Change Control Board, with approval thresholds and a worked change log.
- **Communication and quality management (Sections 11 and 12):** a communication matrix for each stakeholder group and measurable quality targets based on ISO/IEC 25010.

Section 13 explains how GitHub will act as the configuration management tool. It covers the repository structure, branching strategy, pull request reviews, GitHub Actions, release tags and how change requests are traced to code. Team collaboration is managed on a GitHub Projects board (Appendix B).

1. Introduction

Businesses increasingly compete on how quickly they can act on data. Few [1] defines a dashboard as a visual display of the most important information needed to achieve objectives, arranged on a single screen so it can be monitored at a glance. When that information is refreshed continuously rather than weekly, managers can react to events while they still matter, for example by moving stock to a store that is selling out.

Delivering such a dashboard is not just a reporting task. It involves integrating several operational systems, building a reliable data pipeline, protecting customer data and changing how people work. Schwalbe [2] notes that IT projects often fail because of unclear scope, poor schedule control and weak handling of change, rather than because of the technology itself. This plan therefore follows the PMBOK Guide [3] and puts its effort into scope definition, dependency-based scheduling, risk management and controlled change.

The report is organised as follows. Section 2 reviews the relevant literature. Section 3 describes the business case and Section 4 presents the project charter. Sections 5 to 8 define the scope, work breakdown structure, schedule and resource allocation. Sections 9 and 10 set out the risk, issues and change control processes, Sections 11 and 12 cover communication and quality management, and Section 13 describes configuration management with GitHub. Section 14 concludes.

2. Literature Review

This section reviews the research and professional standards that shaped the plan. It covers the performance of IT projects and the frameworks used to manage them, research on business intelligence and real-time analytics, and established practice in risk, change, configuration and quality management. Section 2.5 summarises how these findings were applied.

2.1 IT project performance

IT projects have a poor record of delivering on time and on budget. Flyvbjerg and Budzier [4] analysed 1,471 IT projects and found an average cost overrun of 27%, but the average hid a long tail: one project in six was a "black swan" with an average cost overrun of 200% and a schedule overrun of almost 70%. They conclude that IT projects should be stress-tested for extreme outcomes rather than planned for the average case. Kappelman et al. [5] reached a related conclusion from a survey of IT project experts. None of the twelve most important early warning signs of failure they identified was purely technical. The list was dominated by people and process problems such as weak top management support, poor stakeholder involvement, undocumented requirements and success criteria, no change control process and communication breakdowns between stakeholders. This suggests that most of the risk in a project like this one sits in governance, scope and communication rather than in the cloud technology.

2.2 Project management frameworks

The main frameworks respond to these problems in different ways. The PMBOK Guide [3] now describes project management through twelve principles and eight performance domains rather than prescriptive process groups, and expects each project to tailor its approach. PRINCE2 [6] is more prescriptive. It requires continued business justification, management by stages and management by exception, where each level of management works within agreed tolerances and escalates only when they are forecast to be exceeded. Kerzner [7] and Schwalbe [2] both stress that integration, scope and schedule management are where IT projects most often go wrong. Agile approaches such as Scrum [8] take a different view, accepting that requirements will change and delivering in short, inspected increments, but they say little about fixed budgets or regulatory deadlines.

No single framework fits this project. Harbourline has a fixed budget and a hard go-live date before the November peak, which favours predictive planning and stage gates. Dashboard requirements, however, tend to change once users see real data, which favours iterative delivery. The plan therefore uses a hybrid approach: PRINCE2-style stages and tolerances at the governance level (milestones M1 to M4 and the change approval thresholds), and two-week sprints managed on a GitHub Projects board within the build phases.

2.3 Business intelligence and real-time analytics

Chen et al. [9] describe how business intelligence has moved from periodic reporting on structured data towards continuous analysis of large, fast-moving data, and note that its value comes from shortening the time between an event and a decision. The architecture in this plan follows established patterns for doing this. Kreps et al. [10] showed that a log-based messaging system can capture high

volumes of events with low latency, which is the model Azure Event Hubs follows. Marz and Warren [11] argue that real-time systems should combine a fast "speed layer" with a slower but complete batch layer; the plan's mix of streaming for POS and e-commerce data and 15-minute batch loads for ERP and CRM data is a practical version of that design. Kimball and Ross [12] recommend dimensional (star schema) models with conformed dimensions so that every report uses the same definition of each measure, which is the basis for the KPI dictionary.

Technology alone does not guarantee success. In a study of data warehouse projects, Wixom and Watson [13] found that management support, user participation and the quality of source systems shaped the data quality and system quality of the result, which in turn shaped its perceived benefits. Few [1] adds that many dashboards fail because they are cluttered or use the wrong displays, not because the data is wrong. Davis [14] showed that people adopt a system when they find it useful and easy to use. Together these studies explain why the plan invests in data profiling, user involvement in wireframes and UAT, and training, and why poor data quality (R1) and low adoption (R6) are treated as major risks.

2.4 Risk, change, configuration and quality management

Boehm [15] defined risk exposure as the probability of an unsatisfactory outcome multiplied by the loss it would cause, and recommended resolving the largest risks early through prototypes and benchmarks. ISO 31000 [16] adds that risk management should be part of governance, iterative and supported by ongoing communication. The plan's likelihood × impact scoring and the early POS streaming proof-of-concept apply these ideas directly. For change, both the PMBOK Guide [3] and PRINCE2 [6] require every change to an approved baseline to be assessed for impact and approved by an authority with delegated limits, which answers the "no change control process" warning sign identified by Kappelman et al. [5].

IEEE Std 828 [17] defines configuration management as four activities: identifying configuration items, controlling changes to them, recording their status and auditing them. Git [18] and platforms built on it, such as GitHub, support all four, and Kim et al. [19] show that version control combined with peer review and automated testing in a deployment pipeline improves both delivery speed and reliability. For quality, ISO 9001 [20] promotes a plan–do–check–act cycle, and ISO/IEC 25010 [21] provides a model of product quality characteristics, such as functional suitability, performance efficiency, usability, reliability and security, that can be turned into measurable targets.

2.5 Implications for this plan

Table 1 summarises how the literature has shaped the plan.

Finding from the literature	Sources	Response in this plan	Section
Most IT failures stem from governance, scope, change and communication rather than technology	[5], [4]	Signed charter, scope baseline, Change Control Board and communication plan	4, 5, 10, 11
IT projects carry a long tail of cost and schedule overruns	[4], [15]	10% contingency, milestone gates and an early proof-of-concept on the largest technical risk	5, 7, 9

Finding from the literature	Sources	Response in this plan	Section
Fixed dates need predictive control; changing requirements need iteration	[6], [8]	Hybrid approach: milestone gates with two-week sprints on GitHub Projects	7, 11
Data quality, user participation, usefulness and ease of use drive success and adoption	[13], [14], [1]	Data profiling and quality rules, users in wireframes and UAT, pilot stores, training	6, 9, 12
Real-time systems combine streaming and batch layers on a governed data model	[10], [11], [12]	Event Hubs streaming, 15-minute batch loads, star schema and KPI dictionary	3
Configuration and quality need defined processes and measurable targets	[17], [21]	GitHub configuration management and ISO/IEC 25010 quality metrics	12, 13

Table 1. How the literature informs the plan

3. Project Background and Business Case

3.1 Current situation

Harbourline's data is spread across four systems that do not share information:

Source system	Data held	Current access
POS system (40 stores)	Sales transactions, returns, discounts	End-of-day CSV export
ERP (finance and inventory)	Stock levels, purchase orders, product master	Weekly report run by IT
E-commerce platform	Online orders, cart abandonment, web traffic	Vendor portal only
CRM / loyalty program	Customer profiles, loyalty points, segments	Manual extract on request

Table 2. Current data sources

Each Monday, two finance analysts merge these extracts in Excel and email PDF reports to managers. The problems this causes are:

- **Decision latency:** data is two to seven days old by the time it is used.
- **Stock-outs and overstock:** fast-selling lines are noticed too late to be replenished, and slow lines are discounted late.
- **Inconsistent figures:** different teams calculate "net sales" and "gross margin" differently, which leads to disputes.
- **Manual effort:** about 30 staff hours per week go into report preparation.

3.2 Proposed solution

The project will deliver a cloud-based analytics platform on Microsoft Azure. Streaming data (POS and online orders) will be captured by **Azure Event Hubs**, which is compatible with the Apache Kafka log-based messaging model [10], and processed by **Azure Stream Analytics**. Batch data from the ERP and CRM will be loaded by **Azure Data Factory** every 15 minutes. All data will land in an **Azure Synapse Analytics** warehouse built as a dimensional (star schema) model [12], and will be presented in **Power BI** dashboards for three user groups.

Layer	Technology	Purpose
Ingestion (streaming)	Azure Event Hubs + Stream Analytics	Capture POS and online transactions within seconds
Ingestion (batch)	Azure Data Factory	Load ERP and CRM changes every 15 minutes
Storage and modelling	Azure Data Lake + Synapse Analytics	Governed star-schema warehouse and KPI definitions
Presentation	Power BI (DirectQuery + aggregations)	Role-based dashboards, alerts, mobile view
Security	Microsoft Entra ID, row-level security	Single sign-on, MFA, store-level data access
DevOps	GitHub + GitHub Actions	Version control, code review, automated deployment

Table 3. Solution architecture overview

3.3 Expected benefits

- KPIs refreshed in five minutes or less, instead of weekly.
- A target 15% fall in stock-outs on the top 500 products within six months of go-live.
- About 30 hours per week of analyst time released from manual reporting (roughly AUD $70,000 per year).
- One agreed set of KPI definitions across the business.

4. Project Charter

A project charter formally authorises a project and gives the project manager authority to apply organisational resources to it [3]. The charter below was prepared in activity 1.1 and signed by the sponsor on 12 February 2027. It summarises the project at a high level; the detailed scope baseline is in Section 5.

4.1 Charter summary

Charter item	Detail
Project title	Real-Time Business Insights: Data Analytics Dashboard
Project sponsor	Chief Operating Officer (COO), Harbourline Retail Group
Project manager	Project manager (contract), reporting to the sponsor
Business need	Store, inventory and executive decisions are made on sales and stock data that is up to seven days old and assembled by hand from four systems.
Purpose	Deliver a cloud analytics platform and role-based Power BI dashboards that show sales, inventory and customer KPIs within five minutes of each transaction.
Objectives	O1 to O5 in Section 5.1: five-minute latency, four integrated sources, three adopted dashboards, 30 hours a week of manual effort removed, on time and on budget.
High-level requirements	POS, ERP, e-commerce and CRM data integrated; role-based dashboards with stock alerts; row-level security; data held in Australian Azure regions; Privacy Act compliance.
Duration and milestones	1 February to 3 September 2027 (31 weeks). M1 plan baseline 26 Mar, M2 UAT sign-off 16 Jul, M3 go-live 13 Aug, M4 closure 3 Sep 2027.
Budget	AUD $452,760, including a $41,160 contingency reserve controlled by the sponsor.
Key risks	Source data quality, legacy POS streaming, user adoption, privacy of customer data (Section 9).
Success criteria	Acceptance criteria in Section 5.5 met and signed off by the sponsor and the business representatives of each user group.
Project manager authority	Assign team members within the approved plan; approve minor changes (one week or less, $5,000 or less, within float); escalate anything beyond these limits.
Approvals	Chief Operating Officer (sponsor), Chief Financial Officer, Head of IT and project manager, 12 February 2027.

Table 4. Project charter summary

4.2 Sponsorship and governance

The Chief Operating Officer is the project sponsor because the main benefits, fewer stock-outs and faster store decisions, fall within store operations, and because the COO has the authority to release funds and resolve conflicts between departments. Kappelman et al. [5] rank weak top management support among the strongest warning signs of failure, so the governance structure and the sponsor's commitments are written into the charter:

- **Steering committee:** chaired by the sponsor, with the CFO, Head of IT, Head of Retail Operations and project manager. It meets monthly and at each milestone gate (M1 to M4) to approve progress against the baseline.

- **Change Control Board:** decides major change requests within the thresholds in Section 10.2.
- **Project manager:** runs the project day to day within the approved tolerances and escalates any forecast breach to the sponsor.
- **Sponsor commitments:** attend every steering committee meeting, decide escalated issues within two business days, and introduce the dashboards to store managers at go-live.

4.3 Project team roles and responsibilities

Table 5 defines each role, its time commitment and its main responsibilities. Every work package in the WBS (Section 6) is owned by one of these roles, and the RACI matrix in Section 8.3 shows how they share responsibility for each deliverable.

Role	Commitment	Key responsibilities	Reports to
Project sponsor (COO)	About 2 hours a week	Owns the business case; chairs the steering committee and CCB; approves baselines and milestones; releases contingency; removes organisational blockers	CEO
Project manager	Full time, weeks 1 to 31	Plans, schedules and controls the project; owns the risk, issue and change logs; runs the project board and status reporting; leads the team	Sponsor
Business analyst	Full time, 12 weeks	Runs requirements and KPI workshops; writes the KPI dictionary and UAT scripts; is the link between the team and the business	Project manager
Solution architect	Part time, weeks 7 to 11 (internal)	Designs the architecture and security; leads the POS streaming proof-of-concept; approves technical design changes	Head of IT
Data engineers (2)	Full time, 20 weeks each	Build the cloud environment, CI/CD, streaming and batch pipelines and data quality rules; support testing and hypercare	Project manager
BI developers (2)	Full time, 12 weeks each	Build the wireframes, warehouse model, dashboards, alerts and row-level security; tune performance	Project manager
Test analyst	Full time, 8 weeks	Writes the test strategy; runs system, performance and security testing; manages the defect log; supports UAT	Project manager
Change and training lead	Full time, 6 weeks	Plans training and store communications; recruits pilot champions; tracks adoption after go-live	Project manager
Head of IT	About 2 hours a week	Owns the Azure platform and security; accepts the handover to BAU support	Sponsor
Privacy officer	As required	Approves the privacy impact assessment and access to customer data	CFO
Business representatives	About 4 hours a week in requirements and UAT	Store managers, inventory and finance staff who provide requirements, test the dashboards and sign off UAT	Their line managers

Table 5. Project team roles and responsibilities

5. Project Scope Statement

The scope statement sets the boundaries of the project and is the baseline against which change requests are assessed [3]. It was developed from the business case and is approved by the project sponsor at Milestone M1.

5.1 Project objectives (SMART)

#	Objective	Measure of success
O1	Reduce reporting latency for sales and inventory KPIs	Data visible in dashboards within 5 minutes of a transaction, by go-live (week 28)
O2	Integrate the four core data sources into one governed warehouse	POS, ERP, e-commerce and CRM loaded; reconciliation accuracy of 98% or better against source totals
O3	Deliver role-based dashboards that people use	3 dashboards live; 80% or more of 150 target users active weekly within 4 weeks of go-live
O4	Reduce manual reporting effort	At least 30 analyst hours per week removed from weekly report preparation
O5	Deliver on time and on budget	Closure by 3 September 2027 within AUD $452,760 (including contingency)

Table 6. Project objectives

5.2 Project deliverables

Deliverable	Description
D1 Project management documents	Charter, this project plan, risk register, issue log, change log, status reports, closure report
D2 Requirements and KPI dictionary	Signed-off business requirements and agreed definitions for 25 KPIs
D3 Solution architecture	Architecture and security design, data flow diagrams, privacy impact assessment
D4 Data platform	Streaming and batch pipelines, data lake, Synapse warehouse, data quality rules
D5 Dashboards	Executive, Store Manager and Inventory dashboards with stock alerts and mobile layouts
D6 Test evidence	System, performance, security and UAT results with defect log
D7 Training and support	Training sessions for 150 users, user guides, support runbook, handover to BAU support team
D8 Configuration repository	GitHub repository containing all code, models, dashboard files and documentation, with tagged releases

Table 7. Project deliverables

5.3 In scope

- Integration of POS, ERP (sales, stock, product master), e-commerce and CRM loyalty data.
- Near real-time ingestion (5 minutes or less) for sales and stock movements; 15-minute batch loads for ERP and CRM.

- Three Power BI dashboards: Executive (company-wide KPIs), Store Manager (store-level sales, staffing and stock), and Inventory (stock levels, reorder alerts, supplier lead times).
- Email and mobile push alerts when stock of a top-500 product falls below its reorder point.
- Row-level security so each store manager only sees their own store.
- Pilot in 5 stores, then rollout to all 40 stores and head office.
- Training, user documentation and one month of hypercare support.

5.4 Out of scope (exclusions)

- Replacing or upgrading the ERP, POS or CRM systems.
- Predictive analytics, machine-learning forecasting or AI-driven recommendations (proposed for a later phase).
- Supplier and logistics partner data feeds.
- A custom native mobile app (the Power BI mobile app will be used).
- Ongoing BAU support after the hypercare period (handed to the internal IT support team).

5.5 Acceptance criteria

- Every in-scope KPI matches the source system within a 2% tolerance in reconciliation tests.
- Dashboard pages load in 3 seconds or less for 95% of requests at three times normal peak load.
- End-to-end data latency is 5 minutes or less for 99% of POS transactions over a 7-day test window.
- No critical or high-severity defects open at UAT sign-off; security testing passed.
- Sign-off from the sponsor and the business representatives of each user group.

5.6 Assumptions

- The POS vendor provides an API or change-data-capture (CDC) feed, or allows a store-level agent to be installed.
- Harbourline's existing Microsoft 365 tenancy and Entra ID accounts are used for sign-in.
- Business subject matter experts are available for about 4 hours a week during requirements and UAT.
- Store network bandwidth is sufficient for streaming transaction events.

5.7 Constraints

- **Budget:** AUD $452,760, including 10% contingency (see Table 8).
- **Time:** go-live must happen before the November peak trading season, so the deadline is mid-August 2027.
- **Compliance:** customer data must be handled under the *Privacy Act 1988 (Cth) [22]* and the Australian Privacy Principles [23]. Data stays in Australian Azure regions.
- **Change freeze:** no deployments to stores during public holidays or end-of-month stocktakes.

5.8 Budget estimate

Cost item	Basis of estimate	Cost (AUD)
Project manager	31 weeks × $3,200	99,200
Business analyst	12 weeks × $2,800	33,600
Data engineers (2)	2 × 20 weeks × $3,000	120,000
BI developers (2)	2 × 12 weeks × $2,800	67,200
Test analyst	8 weeks × $2,400	19,200
Change and training lead	6 weeks × $2,400	14,400
Azure consumption and Power BI licences	Build period and first 3 months of use	46,000
Training materials and travel to stores	Estimate	12,000
Subtotal		**411,600**
Contingency reserve (10%)	Controlled by sponsor through change process	41,160
Total project budget		**452,760**

Table 8. Budget estimate

5.9 Key stakeholders

Stakeholder	Role in project	Interest / influence
Chief Operating Officer	Project sponsor; chairs steering committee and Change Control Board	High / High
Project manager	Plans, monitors and controls the project; owns risk and change logs	High / High
Head of IT	Owns cloud platform, security and BAU support	High / High
Store managers (40)	Primary dashboard users; pilot and UAT participants	High / Medium
Inventory and merchandise team	Users of inventory dashboard and stock alerts	High / Medium
Finance (CFO and analysts)	KPI definitions; budget oversight	Medium / High
Privacy officer	Approves privacy impact assessment and data access	Medium / High
POS and e-commerce vendors	Provide data feeds and APIs	Low / Medium

Table 9. Stakeholder register (summary)

6. Work Breakdown Structure

The work breakdown structure (WBS) breaks the total scope into seven phases and 23 work packages (Table 10). Each work package has a single owner and a clear output, which makes it easier to estimate and assign. Project management (WBS 8) runs for the whole project. Milestones M1 to M4 are decision points where the steering committee reviews progress before work moves on.

WBS	Phase / work package	Output	Owner
1	**Initiation**		
1.1	Project charter and stakeholder analysis	Approved charter, stakeholder register	Project manager
1.2	Kick-off and team onboarding	Kick-off pack, team access	Project manager
2	**Planning and requirements**		
2.1	Business requirements and KPI workshops	Requirements document, KPI dictionary	Business analyst
2.2	Data source assessment	Data profiling report	Data engineer
2.3	Solution architecture and security design	Architecture document, privacy impact assessment	Solution architect
3	**Build: data platform**		
3.1	Cloud environment and GitHub CI/CD setup	Dev/test/prod environments, pipelines	Data engineer
3.2	Streaming ingestion (POS and e-commerce)	Event Hubs and Stream Analytics jobs	Data engineer
3.3	Batch ingestion (ERP and CRM)	Data Factory pipelines	Data engineer
3.4	Data warehouse and semantic model	Star schema, Power BI dataset	BI developer
3.5	Data quality rules and validation	Automated DQ checks, reconciliation report	Data engineer
4	**Build: dashboards**		
4.1	Dashboard wireframes and prototype	Approved wireframes	BI developer
4.2 to 4.4	Executive, Store Manager and Inventory dashboards	Three dashboards with alerts	BI developers
4.5	Row-level security and access roles	Security roles mapped to Entra ID groups	BI developer
5	**Testing**		
5.1 to 5.3	System/performance, security/privacy, UAT	Test reports, UAT sign-off	Test analyst
6	**Deployment**		
6.1 to 6.3	Pilot (5 stores), training, rollout (40 stores)	Live system, trained users	Change and training lead
7	**Closure**		
7.1 to 7.2	Hypercare, BAU handover, lessons learned	Handover pack, closure report	Project manager

WBS	Phase / work package	Output	Owner
8	**Project management (ongoing)**	Status reports, risk/issue/change logs	Project manager

Table 10. Work breakdown structure

7. Project Schedule and Gantt Chart

The schedule was built with the critical path method (CPM). Durations were estimated in weeks from the team's experience and the effort shown in the budget. Each activity was then linked to the activities it depends on with finish-to-start dependencies, meaning an activity cannot begin until all its predecessors are complete. A forward pass gives the earliest start and finish of each activity, and a backward pass gives the latest dates. The difference is the **total float**, the time an activity can slip without delaying the project end date [2].

7.1 Activity list and dependencies

ID	Activity	Duration	Predecessors	Start	Finish	Total float
1.1	Project charter & stakeholder analysis	2 wk	None	01 Feb 2027	12 Feb 2027	0 (critical)
1.2	Kick-off & team onboarding	1 wk	1.1	15 Feb 2027	19 Feb 2027	0 (critical)
2.1	Business requirements & KPI workshops	3 wk	1.2	22 Feb 2027	12 Mar 2027	0 (critical)
2.2	Data source assessment	2 wk	1.2	22 Feb 2027	05 Mar 2027	1 wk
2.3	Solution architecture & security design	2 wk	2.1, 2.2	15 Mar 2027	26 Mar 2027	0 (critical)
M1	Plan baseline approved	Milestone	2.3	26 Mar 2027		0 (critical)
3.1	Cloud environment & GitHub CI/CD setup	2 wk	M1	29 Mar 2027	09 Apr 2027	0 (critical)
3.2	Streaming ingestion (POS & e-commerce)	4 wk	3.1	12 Apr 2027	07 May 2027	0 (critical)
3.3	Batch ingestion (ERP & CRM)	3 wk	3.1	12 Apr 2027	30 Apr 2027	1 wk
3.4	Data warehouse & semantic model	4 wk	3.1	12 Apr 2027	07 May 2027	0 (critical)
3.5	Data quality rules & validation	2 wk	3.2, 3.3, 3.4	10 May 2027	21 May 2027	0 (critical)
4.1	Dashboard wireframes & prototype	2 wk	2.1	15 Mar 2027	26 Mar 2027	8 wk
4.2	Executive dashboard	3 wk	3.5, 4.1	24 May 2027	11 Jun 2027	0 (critical)
4.3	Store manager dashboard	3 wk	3.5, 4.1	24 May 2027	11 Jun 2027	0 (critical)
4.4	Inventory dashboard & alerts	3 wk	3.5, 4.1	24 May 2027	11 Jun 2027	0 (critical)
4.5	Row-level security & access roles	1 wk	4.2, 4.3, 4.4	14 Jun 2027	18 Jun 2027	0 (critical)
5.1	System & performance testing	2 wk	4.5	21 Jun 2027	02 Jul 2027	0 (critical)
5.2	Security & privacy testing	1 wk	4.5	21 Jun 2027	25 Jun 2027	1 wk
5.3	User acceptance testing	2 wk	5.1, 5.2	05 Jul 2027	16 Jul 2027	0 (critical)
M2	UAT sign-off	Milestone	5.3	16 Jul 2027		0 (critical)
6.1	Pilot in 5 stores	2 wk	M2	19 Jul 2027	30 Jul 2027	0 (critical)
6.2	User training & documentation	2 wk	M2	19 Jul 2027	30 Jul 2027	0 (critical)
6.3	Rollout to all 40 stores	2 wk	6.1, 6.2	02 Aug 2027	13 Aug 2027	0 (critical)
M3	Go-live	Milestone	6.3	13 Aug 2027		0 (critical)
7.1	Hypercare & handover to BAU	2 wk	M3	16 Aug 2027	27 Aug 2027	0 (critical)
7.2	Lessons learned & project closure	1 wk	7.1	30 Aug 2027	03 Sep 2027	0 (critical)
M4	Project closed	Milestone	7.2	03 Sep 2027		0 (critical)

Table 11. Activity list with dependencies and float (start Monday 1 February 2027; weeks run Monday to Friday)

7.2 Key dependencies

The most important dependencies that shape the schedule are:

- **Requirements before architecture (2.1, 2.2 → 2.3):** the architecture cannot be designed until the KPIs and data sources are known.
- **Plan baseline before build (M1 → 3.1):** no build work starts until the sponsor approves the scope, schedule and budget.
- **Data platform before dashboards (3.5 → 4.2 to 4.4):** dashboards need validated data. Wireframes (4.1) can be done early because they only depend on requirements, so 4.1 has 8 weeks of float.
- **Parallel ingestion (3.2, 3.3, 3.4):** streaming, batch and warehouse work run at the same time after the environment is ready, which saves about 7 weeks compared with doing them in sequence.
- **Testing gates (5.3 → M2 → 6.1):** the pilot only begins after UAT sign-off.
- **Pilot and training before full rollout (6.1, 6.2 → 6.3):** lessons from the pilot stores are built into the rollout.

7.3 Gantt chart

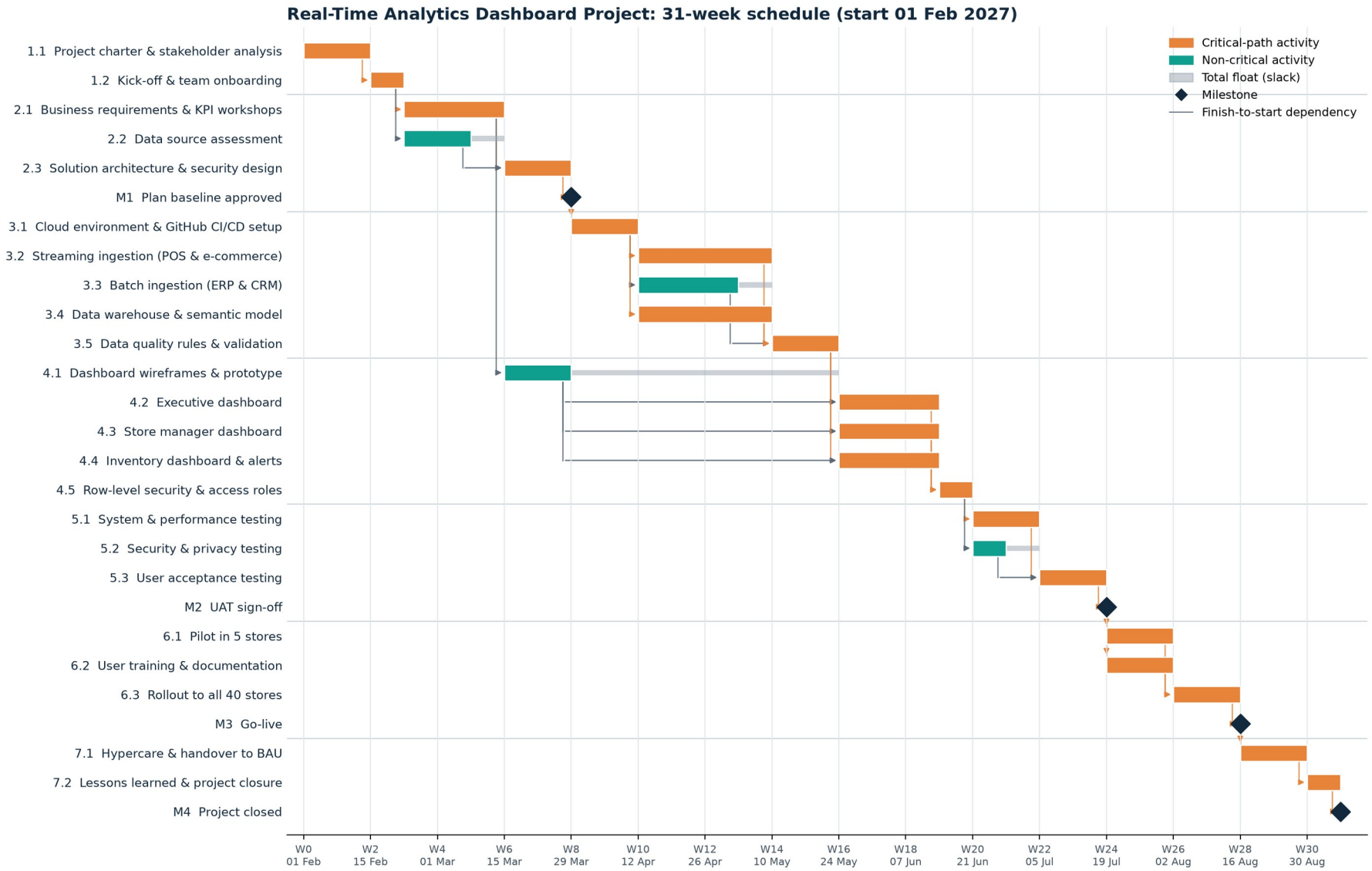


Figure 1. Gantt chart showing finish-to-start dependencies, critical path (orange), float (grey) and milestones (diamonds)

7.4 Critical path and milestones

The critical path is **1.1 → 1.2 → 2.1 → 2.3 → M1 → 3.1 → 3.2 / 3.4 → 3.5 → 4.2 / 4.3 / 4.4 → 4.5 → 5.1 → 5.3 → M2 → 6.1 / 6.2 → 6.3 → M3 → 7.1 → 7.2 → M4**, with a total length of **31 weeks**. Any delay to these activities delays go-live, so they get the most attention in weekly status reviews. Activities with float (2.2, 3.3, 4.1 and 5.2) can absorb small delays, and their resources can be moved to critical work if needed.

Milestone	Description	Target date	Approver
M1	Project plan baseline approved (scope, schedule, budget)	26 Mar 2027	Sponsor / steering committee
M2	UAT sign-off; no critical or high defects open	16 Jul 2027	Business representatives
M3	Go-live in all 40 stores	13 Aug 2027	Sponsor
M4	Project closed; handover to BAU accepted	03 Sep 2027	Sponsor and Head of IT

Table 12. Project milestones

Go-live (M3) is planned for mid-August 2027. This leaves about 11 weeks before the November peak season, so the time constraint in Section 5.7 is met. The 10% contingency reserve and the change control process (Section 10) protect this date.

8. Resource Allocation and Responsibility Matrix

Resources were allocated from the activity list in Section 7.1. Each role was assigned to the activities it owns, and the weeks in which it is needed were taken from the schedule. Table 13 shows the allocation and links it to the budget in Section 5.8 (week 1 begins on 1 February 2027).

8.1 Resource allocation

Role	Qty	Assigned activities	Weeks on project	Person-weeks	Cost (AUD)
Project manager	1	All (WBS 1 to 8)	1 to 31	31	99,200
Business analyst	1	2.1, 2.3, 4.1, 5.3, 6.1, 6.2	4 to 9, 21 to 26	12	33,600
Data engineers	2	2.2, 2.3, 3.1 to 3.5, 5.1, 6.1, 6.3, 7.1	4 to 5, 7 to 16, 21 to 22, 25 to 30	40	120,000
BI developers	2	4.1, 3.4, 4.2 to 4.5	7 to 8, 11 to 20	24	67,200
Test analyst	1	Test planning in 3.5; 5.1 to 5.3	15 to 16, 19 to 24	8	19,200
Change and training lead	1	5.3, 6.1 to 6.3	23 to 28	6	14,400
Solution architect	1	2.3, 3.1, POS proof-of-concept in 3.2	7 to 11, part time	Internal	Not charged
Business representatives	–	2.1, 4.1, 5.3, 6.1	About 4 hours a week	Internal	Not charged
Total labour				**121**	**353,600**

Table 13. Resource allocation by role (labour costs match Table 8)

8.2 Resource loading and levelling

The team is largest when the build and testing phases overlap. Six people work on the project in weeks 7 to 8 (architecture and wireframes) and weeks 15 to 16 (data quality validation and test planning), and the team falls to three people during hypercare. Three decisions keep the plan free of over-allocation:

- **Parallel work is split across roles.** Streaming ingestion (3.2) and batch ingestion (3.3) are each done by one data engineer, while the warehouse and semantic model (3.4) is built by the two BI developers, so no one works on two activities at once.
- **Float is used to smooth demand.** The wireframes (4.1) are done in weeks 7 to 8, at the start of their float window, so the BI developers finish them before the warehouse build begins in week 11.
- **Specialists join only when needed.** The test analyst starts in week 15 to prepare test plans, and the change and training lead joins in week 23 for UAT and rollout, rather than both being funded for the whole project.

8.3 Responsibility assignment (RACI) matrix

The RACI matrix in Table 14 shows who is Responsible for doing the work, who is Accountable for the result, who must be Consulted and who must be kept Informed [3]. Each deliverable has exactly one accountable role, so there is never doubt about who makes the final decision.

Deliverable (WBS)	Sponsor	PM	BA	Arch.	Data eng.	BI dev.	Test	Change lead	Business reps
Charter and plan baseline (1.1, M1)	A	R	C	C	I	I	I	I	I
Requirements and KPI dictionary (2.1)	C	A	R	C	C	C	I	I	C
Architecture and security design (2.3)	I	A	C	R	C	C	I	–	–
Data platform build (3.1 to 3.5)	–	A	C	C	R	R	C	–	–
Dashboards and row-level security (4.1 to 4.5)	I	A	C	C	C	R	C	I	C
Testing and UAT (5.1 to 5.3)	I	A	C	C	C	C	R	I	R
Pilot, training and rollout (6.1 to 6.3)	I	A	C	–	C	C	–	R	C
Risk, issue and change logs (8)	C	A/R	C	C	C	C	C	C	I
Change request decisions (CCB)	A	R	C	C	I	I	I	I	C
Configuration management (GitHub)	–	A	I	C	R	R	C	–	–
Hypercare, handover and closure (7.1, 7.2)	A	R	I	C	C	C	I	C	I

Table 14. RACI matrix (R = responsible, A = accountable, C = consulted, I = informed, – = not involved)

9. Risk and Issues Management Strategy

A **risk** is an uncertain event that, if it happens, affects a project objective. An **issue** is something that is already happening and needs action now. The project manages both, using separate logs and separate processes, based on ISO 31000:2018 [16] and the PMBOK Guide [3].

9.1 Risk management process

1. **Identify:** risks are collected in the kick-off workshop, through a review of assumptions and constraints, from lessons learned on earlier IT projects, and by any team member at any time through a GitHub issue labelled *risk*.
2. **Analyse:** each risk is scored for likelihood (1 to 5) and impact (1 to 5) on time, cost, quality or compliance. The score (L × I) places it on the risk matrix (Figure 2).
3. **Plan the response:** each risk gets an owner and one of four responses: *avoid* (change the plan to remove the risk), *mitigate* (reduce likelihood or impact), *transfer* (move it to a third party through a contract or insurance) or *accept* (with a contingency plan).
4. **Monitor and control:** the risk register is reviewed weekly by the project team and monthly by the steering committee. Each risk has a trigger, and when the trigger occurs the risk becomes an issue.
5. **Communicate:** the top five risks are included in every fortnightly status report.

9.2 Risk assessment matrix

Likelihood ↓ / Impact →	1 Negligible	2 Minor	3 Moderate	4 Major	5 Severe
5 Almost certain	5	10	15	20	25
4 Likely	4	8	12 **R3, R9**	16 **R1**	20
3 Possible	3	6	9 **R5**	12 **R6, R8**	15 **R2**
2 Unlikely	2	4	6	8 **R7**	10 **R4**
1 Rare	1	2	3	4	5

Figure 2. 5 × 5 risk matrix showing where each register risk sits. Ratings: Low 1 to 4, Medium 5 to 9, High 10 to 14, Extreme 15 to 25

Rating	Score	Required action
Extreme	15 to 25	Report to sponsor within 24 hours; response plan approved by steering committee
High	10 to 14	Active mitigation plan; reviewed weekly by the project manager
Medium	5 to 9	Owner monitors; reviewed fortnightly
Low	1 to 4	Accept and monitor in the register

Table 15. Risk rating thresholds and actions

9.3 Risk register

ID	Risk description	Category	L	I	Score	Response strategy	Owner
R1	Poor source data quality (duplicate SKUs, missing store codes) causes wrong KPIs	Data	4	4	16 Extreme	Mitigate: profile data in 2.2; automated DQ rules in 3.5; assign a business data owner for each source	Data engineering lead
R2	Legacy POS cannot stream events in real time	Technical	3	5	15 Extreme	Mitigate: proof-of-concept at the start of 3.2 (week 11); fallback to 5-minute CDC micro-batches	Solution architect
R3	Stakeholders disagree on KPI definitions, delaying sign-off	Stakeholder	4	3	12 High	Mitigate: KPI dictionary signed off at M1; CFO decides disputes	Business analyst
R4	Customer personal data is exposed through dashboards	Compliance	2	5	10 High	Avoid: no personal identifiers in dashboards; masking, MFA, row-level security; privacy impact assessment	Privacy officer
R5	Cloud consumption costs exceed the forecast	Financial	3	3	9 Medium	Mitigate: Azure budgets and alerts at 80%; autoscale limits; monthly cost review	Project manager
R6	Low adoption by store managers	Organisational	3	4	12 High	Mitigate: managers in UAT; pilot-store champions; training; mobile layouts	Change and training lead
R7	Key data engineer leaves during the build	Resource	2	4	8 Medium	Mitigate: code and runbooks in GitHub; pair programming. Transfer: vendor backfill clause	Project manager
R8	Dashboards slow down during peak trading	Technical	3	4	12 High	Mitigate: load test at 3× peak in 5.1; pre-calculated aggregations	BI lead
R9	Scope creep from new report requests	Scope	4	3	12 High	Mitigate: all requests go through change control (Section 10)	Project manager

Table 16. Project risk register (L = likelihood, I = impact)

The two highest risks (R1 data quality and R2 POS streaming) are both technical and both affect the critical path. They are handled early on purpose. Data profiling in week 4 and a streaming proof-of-concept in week 11 show whether either risk is real while there is still time to change the approach without missing go-live.

9.4 Issues management

Any team member can raise an issue as a GitHub issue using the *issue* template. The project manager reviews new issues every day and assigns a priority and an owner. Issues are tracked to closure on the GitHub Projects board, and the issue log is summarised in the fortnightly status report.

Priority	Definition	Response time	Escalation
P1 Critical	Stops critical-path work or threatens a milestone, the budget or compliance	Same day	PM → sponsor within 24 hours
P2 High	Affects a deliverable or causes a delay that can be absorbed by float	2 business days	Team lead → PM; steering committee if not resolved in 5 days
P3 Normal	Minor impact; team can resolve	Within the sprint	Team lead

Table 17. Issue priorities and escalation path

The escalation path is: **team member → work-package owner → project manager → steering committee → sponsor**. An issue that requires a change to scope, schedule or budget is turned into a change request (Section 10). Table 18 shows example entries from the issue log.

ID	Issue	Priority	Owner	Action / status
I-01	Two pilot stores send POS data with the wrong time zone	P2	Data engineer	Time zone conversion added to stream job; closed
I-02	Privacy officer unavailable to sign the privacy impact assessment before M1	P1	Project manager	Escalated to sponsor; delegate approved; closed
I-03	Power BI licence purchase delayed by procurement	P2	Head of IT	Trial licences used for UAT; purchase order raised; open

Table 18. Sample issue log entries

10. Change Request Management Process

Change is normal in analytics projects. Once users see their data, they often ask for new measures, filters or data sources. Uncontrolled change is the most common cause of scope creep (risk R9). Every change to an approved baseline (scope, schedule, budget or the configuration items in Section 13) must therefore follow the process below. This is the integrated change control process described in the PMBOK Guide [3].

10.1 Change control process

Step	Activity	Responsible	Output
1. Submit	Requester fills in the change request form (GitHub issue template *change-request*): description, reason, business benefit, urgency	Any stakeholder	CR raised with ID (CR-nnn)
2. Log and screen	PM checks the CR is complete and not a duplicate, logs it in the change log and confirms receipt within 2 business days	Project manager	CR logged
3. Impact analysis	Team assesses the effect on scope, schedule (critical path and float), cost, quality, risk and other configuration items	Work-package owner	Impact assessment
4. Review and decide	Decision made according to the approval thresholds in Table 20: approve, reject or defer	PM / CCB / sponsor	Decision recorded
5. Communicate	Requester and stakeholders told of the decision and its reason	Project manager	Updated change log
6. Implement	Baselines (scope, Gantt, budget) updated; work done on a GitHub branch linked to the CR; pull request reviewed	Delivery team	Updated baselines and code
7. Verify and close	Tester confirms the change works; PM closes the CR and records the actual impact	Test analyst / PM	CR closed

Table 19. Seven-step change request process

10.2 Change Control Board and approval thresholds

The **Change Control Board (CCB)** is made up of the sponsor (chair), the project manager, the Head of IT, the BI lead and a business representative. It meets every fortnight, and within 48 hours for urgent requests. Small changes are approved by the project manager so the board is not slowed down by minor items.

Level	Change impact	Approved by
Minor	1 week or less of effort, $5,000 or less, uses available float, no milestone change	Project manager
Major	More than 1 week or more than $5,000, or uses contingency, but no milestone change	Change Control Board
Critical	Changes a milestone date, is over $25,000, or changes the project objectives	Sponsor / steering committee

Table 20. Change approval thresholds

10.3 Change request form

The form is a GitHub issue template, so every change request has an ID, discussion history and links to the code that implements it. Its fields are: CR ID, requester, date, description, reason and benefit, urgency, affected deliverables or configuration items, estimated schedule impact, estimated cost impact, risk impact, recommendation, decision, approver and decision date.

10.4 Example change log

CR	Request	Impact analysis	Decision
CR-001	Add supplier delivery data to the Inventory dashboard (Inventory manager)	New data source; +3 weeks on critical path; +$22,000; outside current scope (Section 5.4)	**Deferred** to Phase 2 by CCB
CR-002	Refresh dashboards every 1 minute instead of 5 (Executive team)	Needs Power BI Premium capacity; +$18,000 per year running cost; small business benefit	**Rejected** by CCB; 5-minute refresh kept
CR-003	Mobile-optimised layout for the Store Manager dashboard (Store managers)	4 days of effort in parallel with 4.3; $3,800; no milestone impact	**Approved** by PM; done on branch feature/CR-003-mobile-layout
CR-004	Add a gross margin by category KPI (CFO)	KPI dictionary and data model change; 3 days; $2,500	**Approved** by PM; released in v1.1.0

Table 21. Example change log

11. Communication Management Plan

Kappelman et al. [5] identify communication breakdown between stakeholders as one of the main early warning signs of project failure. This plan makes sure each stakeholder group receives the information it needs, at the right level of detail and frequency, through an agreed channel [3]. It follows three rules: there is one source of truth for each type of information, bad news is escalated quickly using the issue priorities in Section 9.4, and every regular report has a named owner.

11.1 Communication matrix

Communication	Audience	Purpose and content	Frequency	Channel	Owner
Project kick-off	All stakeholders	Objectives, scope, roles and ways of working	Once (week 3)	Workshop, recorded in Teams	Project manager
Daily stand-up	Delivery team	Progress, plans for the day, blockers	Daily, 15 minutes	Teams call with the project board	Project manager
Sprint review and planning	Team, business analyst, business reps	Demonstrate finished work; agree the next sprint	Every 2 weeks	Teams, GitHub Projects board	Business analyst
Status report	Sponsor, steering committee, Head of IT	Progress against baseline, top five risks, open issues, changes, budget	Fortnightly	Email and project board insights	Project manager
Steering committee	Sponsor, CFO, Head of IT, Head of Retail Ops, PM	Decisions, escalations and milestone gate reviews (M1 to M4)	Monthly and at each milestone	Meeting; minutes in SharePoint	Sponsor
Change Control Board	CCB members	Decide change requests	Fortnightly; within 48 hours if urgent	Teams; GitHub change log	Project manager
Store manager update	40 store managers	What is coming, pilot results, training dates	Monthly; weekly during rollout	Newsletter and Teams channel	Change and training lead
Training sessions	150 dashboard users	How to use the dashboards and alerts	Weeks 25 to 28	Face to face (pilot) and online	Change and training lead
P1 issue escalation	Sponsor	Critical issues that threaten a milestone, budget or compliance	Within 24 hours	Phone, then email	Project manager
Closure report and lessons learned	Sponsor, steering committee	Results against objectives and lessons for future projects	Once (week 31)	Report and workshop	Project manager

Table 22. Communication matrix

11.2 Collaboration tools

- **GitHub Projects:** the team's work management board. Each WBS phase is a milestone and each work package is broken into issues, planned in two-week iterations. The board's progress charts feed the status report. Evidence of the team's use of the board is in Appendix B.

- **GitHub:** code, dashboard files and project documents, together with the risk, issue and change request records that link to them (Section 13). Because the board and the code live in one tool, commits and pull requests are linked automatically to the work item they complete.
- **Microsoft Teams and SharePoint:** meetings, chat channels for each workstream, and signed approvals such as the charter and milestone sign-offs.

11.3 Feedback and escalation

Communication is two-way. Pilot store managers complete a short survey at the end of each pilot week, and sprint reviews give business representatives a regular chance to comment on the dashboards. Concerns that cannot be resolved by the team follow the escalation path in Section 9.4, and any request that would change scope, schedule or budget is raised as a change request (Section 10).

12. Quality Management Approach

Quality means that the deliverables meet their requirements and are fit for use [3]. The approach follows the plan–do–check–act cycle of ISO 9001 [20]: quality targets are set during planning, quality assurance makes sure the team follows good processes, and quality control checks the products themselves. Product quality targets are based on the characteristics in ISO/IEC 25010 [21], with data quality added because it is the largest risk to the project (R1).

12.1 Quality planning: standards and metrics

Quality characteristic	Metric	Target	How measured	When
Functional suitability	KPI values reconciled to source totals	98% or better overall; each KPI within 2%	Automated reconciliation tests	Every load; formally in 5.1
Data quality	Duplicate SKUs; records with a valid store code	No duplicates; 99.5% or more valid	Data quality rules (3.5)	Every load
Performance efficiency	Page load time; end-to-end data latency	3 s or less for 95% of requests at 3× peak; 5 min or less for 99% of POS data	Load test; latency monitoring	5.1; 7-day test window
Reliability	Pipeline availability in trading hours	99.5% or better	Azure Monitor alerts	From the pilot onwards
Security	Open vulnerabilities; row-level security test cases	No critical or high; 100% pass	Security and privacy testing	5.2
Usability	UAT tasks completed unaided; user rating	90% or more; average 4 out of 5 or better	UAT scripts; pilot survey	5.3; 6.1
Maintainability	Changes peer reviewed; data quality rules with automated tests	100% of pull requests; 100% of rules	GitHub branch protection and Actions	Continuous

Table 23. Quality metrics and targets

12.2 Quality assurance

- **Definition of done:** a work item on the project board is only closed when its code has been peer reviewed through a pull request, the automated checks pass, it has been tested in the test environment, its documentation is updated and the business analyst has accepted it.
- **Peer review:** branch protection on GitHub requires one approving review for every change (Section 13.3).
- **Stage-gate reviews:** the Head of IT reviews the architecture before M1, the test exit report is reviewed before M2, and a go-live readiness checklist is completed before M3.
- **Process audit and improvement:** the configuration audits before M2 and M3 (Section 13.4) confirm the process has been followed, and each sprint retrospective records improvements for the next sprint.

12.3 Quality control

- **Testing at every level:** unit tests and SQL checks run in GitHub Actions on every pull request, followed by system and performance testing (5.1), security and privacy testing (5.2) and UAT (5.3).
- **Defect management:** defects are logged as GitHub issues and rated critical, high, medium or low. Critical and high defects must be fixed before release; the exit criterion for M2 is no open critical or high defects.
- **Quality reporting:** open defects by severity, the reconciliation pass rate and data latency are reported in every fortnightly status report, so problems are visible early.

13. Configuration Management Using GitHub

Configuration management makes sure that the project's products, which include code, data models, dashboards and documents, are identified, version-controlled and changed only in a controlled way. It answers three questions at any point: what is the current version, who changed it, and why. It is closely linked to change control, because every approved change request ends up as a recorded change to one or more configuration items.

13.1 Tool selection

We compared three common platforms built on Git, the distributed version control system described by Chacon and Straub [18]:

Criteria	GitHub	GitLab	Bitbucket
Version control	Git	Git	Git
Built-in CI/CD	GitHub Actions	GitLab CI	Bitbucket Pipelines
Issue tracking and boards	Issues, Projects, templates	Issues, boards	Basic, Jira integration
Azure and Power BI integration	Strong (Azure login actions, Microsoft-owned)	Good	Good
Team familiarity and cost	Familiar; Team plan low cost	Less familiar	Less familiar

Table 24. Comparison of configuration management tools

GitHub was selected because it combines version control, issue tracking (used for risks, issues and change requests), project boards and CI/CD in one tool, and it integrates well with Azure.

13.2 Configuration items

Configuration item	Location in repository	Format
Infrastructure definitions	/infrastructure	Bicep templates
Ingestion pipelines and stream jobs	/pipelines	JSON, SQL
Warehouse schema and transformations	/warehouse	SQL scripts
Power BI dashboards and data model	/dashboards	Power BI Project (.pbip) files
Data quality and automated tests	/tests	Python, SQL
Project documents (scope, plan, KPI dictionary, registers)	/docs	Markdown, PDF

Table 25. Configuration items under version control

13.3 Branching strategy and workflow

- **main:** production-ready code only. The branch is protected: direct pushes are blocked, and merges need a pull request, one approving review and passing checks [24].
- **develop:** integration branch deployed automatically to the test environment.
- **feature/<issue-id>-description:** one branch per work item or change request, for example *feature/CR-003-mobile-layout*.

- **release/x.y.z:** stabilisation before UAT or go-live; **hotfix/name:** urgent production fixes branched from main.

Every commit message references its GitHub issue (for example "Add stock alert rule, closes #42"), so each code change can be traced back to a requirement, issue or change request. Pull requests run **GitHub Actions** checks (SQL linting, data-quality unit tests and Bicep validation) before review, and merging to main deploys to production after manual approval.

13.4 Releases, status accounting and audits

- **Baselines and releases:** each milestone is tagged using semantic versioning, for example *v0.1.0* (M1 plan baseline), *v1.0.0* (go-live) and *v1.1.0* (CR-004).
- **Status accounting:** the GitHub Projects board shows each work item, risk, issue and change request as To do, In progress, In review or Done. The release notes list which changes are in each version.
- **Configuration audits:** before M2 and M3 the PM checks that the tagged release matches the approved scope and that every merged change has an approved issue or CR.
- **Rollback:** because every version is kept, a faulty release can be reverted to the previous tag in minutes.

14. Conclusion

This plan sets out how Harbourline Retail Group will move from week-old Excel reports to dashboards refreshed within five minutes, in 31 weeks and within a budget of AUD $452,760. The literature shows that IT projects fail more often through weak governance, scope, change and communication practices than through technology [5], and the plan is built to address those causes. The charter gives the project a committed sponsor, clear governance and a team with defined roles, and the scope statement defines what will and will not be delivered and how success will be measured. The dependency-based Gantt chart identifies the critical path and the activities that have float, and the resource plan and RACI matrix give every work package an owner without over-allocating anyone. The risk and issues strategy deals with the biggest threats, data quality and POS streaming, early in the project and gives a clear escalation path. The change request process and the Change Control Board protect the scope and the go-live date from uncontrolled change. The communication plan keeps each stakeholder group informed at the right frequency, and the quality approach sets measurable targets for data accuracy, performance, security and usability. GitHub ties these together by tracking every configuration item, work item, issue and change request in one place.

The main recommendation is to approve the plan baseline (M1) and start the POS streaming proof-of-concept as early as possible, because it is the largest technical uncertainty on the critical path. A later phase can then add predictive forecasting and supplier data, building on the platform this project delivers.

References

[1] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Burlingame, CA, USA: Analytics Press, 2013.

[2] K. Schwalbe, *Information Technology Project Management*, 9th ed. Boston, MA, USA: Cengage Learning, 2019.

[3] Project Management Institute, *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th ed. Newtown Square, PA, USA: PMI, 2021.

[4] B. Flyvbjerg and A. Budzier, "Why your IT project may be riskier than you think," *Harvard Business Review*, vol. 89, no. 9, pp. 23–25, Sep. 2011.

[5] L. A. Kappelman, R. McKeeman, and L. Zhang, "Early warning signs of IT project failure: The dominant dozen," *Information Systems Management*, vol. 23, no. 4, pp. 31–36, 2006.

[6] AXELOS, *Managing Successful Projects with PRINCE2*, 2017 ed. London, U.K.: TSO, 2017.

[7] H. Kerzner, *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*, 13th ed. Hoboken, NJ, USA: Wiley, 2022.

[8] K. Schwaber and J. Sutherland, "The Scrum Guide," Nov. 2020. [Online]. Available: https://scrumguides.org/scrum-guide.html

[9] H. Chen, R. H. L. Chiang, and V. C. Storey, "Business intelligence and analytics: From big data to big impact," *MIS Quarterly*, vol. 36, no. 4, pp. 1165–1188, Dec. 2012.

[10] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. 6th Int. Workshop Networking Meets Databases (NetDB)*, Athens, Greece, 2011, pp. 1–7.

[11] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY, USA: Manning, 2015.

[12] R. Kimball and M. Ross, *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*, 3rd ed. Indianapolis, IN, USA: Wiley, 2013.

[13] B. H. Wixom and H. J. Watson, "An empirical investigation of the factors affecting data warehousing success," *MIS Quarterly*, vol. 25, no. 1, pp. 17–41, Mar. 2001.

[14] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, Sep. 1989.

[15] B. W. Boehm, "Software risk management: Principles and practices," *IEEE Software*, vol. 8, no. 1, pp. 32–41, Jan. 1991.

[16] *Risk Management — Guidelines*, ISO 31000:2018, International Organization for Standardization, Geneva, Switzerland, 2018.

[17] *IEEE Standard for Configuration Management in Systems and Software Engineering*, IEEE Std 828-2012, 2012.

[18] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Berkeley, CA, USA: Apress, 2014. [Online]. Available: https://git-scm.com/book/en/v2

[19] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. Portland, OR, USA: IT Revolution Press, 2016.

[20] *Quality Management Systems — Requirements*, ISO 9001:2015, International Organization for Standardization, Geneva, Switzerland, 2015.

[21] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.

[22] *Privacy Act 1988* (Cth). [Online]. Available: https://www.legislation.gov.au/C2004A03712/latest

[23] Office of the Australian Information Commissioner, "Australian Privacy Principles guidelines," 2022. [Online]. Available: https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-guidelines

[24] GitHub, "About protected branches," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (accessed Oct. 24, 2026).

Appendix A: Group Contribution

The work was shared equally between the two group members. Both members reviewed the whole report before submission.

Member	Student ID	Report sections	Presentation	Collaboration tools
Gazi Md Tarequr Rahman	68765	Executive summary; 1 Introduction; 2.1 to 2.2 Literature review; 3 Background; 4 Project charter; 5 Scope statement; 6 WBS; 7 Schedule and Gantt chart; 8 Resources and RACI	Slides 1 to 10: introduction, business case, literature, charter, scope, WBS, schedule, resources and RACI	Set up the GitHub Projects board
Atirikta Pokharel	66660	2.3 to 2.5 Literature review; 9 Risk and issues; 10 Change request process; 11 Communication plan; 12 Quality management; 13 Configuration management; 14 Conclusion; references	Slides 11 to 20: risk, issues, change control, communication, quality, GitHub, conclusion	Set up the GitHub repository and branch rules

Table 26. Contribution of each group member

Appendix B: Team Collaboration Evidence (GitHub Projects)

The team used a GitHub Projects board to plan and track its work on this assessment. The board has a milestone for each part of the assessment (project report, presentation and repository) and an issue for each report section and slide group, assigned to the member responsible (Appendix A). Members discussed the work in issue comments and linked each pull request to the issue it completed. The screenshots below show the board in use.

[INSERT SCREENSHOT: GitHub Projects table view listing every work item with its assignee, status and milestone]

Figure B1. Project board table view

[INSERT SCREENSHOT: GitHub Projects board view with items in Todo, In Progress and Done, assigned to both members]

Figure B2. Sprint board showing work in progress

[INSERT SCREENSHOT: one issue opened, showing the assignee, comments between members and the linked pull request]

Figure B3. Task detail showing collaboration between members

[INSERT SCREENSHOT: GitHub Projects roadmap view or Insights chart showing progress over time]

Figure B4. Roadmap and progress

Appendix C: GitHub Repository Evidence

[INSERT GITHUB REPOSITORY URL]

The repository follows the structure and workflow described in Section 13. The screenshots below show it in use.

[INSERT SCREENSHOT: repository home page showing the folders (/infrastructure, /pipelines, /warehouse, /dashboards, /tests, /docs) and the README]

Figure C1. Repository structure

[INSERT SCREENSHOT: commit history with meaningful messages from both members that reference issues]

Figure C2. Commit history

[INSERT SCREENSHOT: branches (main, develop, feature/CR-003-mobile-layout) and the branch protection rule on main]

Figure C3. Branches and branch protection

[INSERT SCREENSHOT: a merged pull request with an approving review and passing GitHub Actions checks]

Figure C4. Reviewed pull request

[INSERT SCREENSHOT: Issues list with risk, issue and change-request labels, and the change request template]

Figure C5. Risks, issues and change requests as GitHub issues

[INSERT SCREENSHOT: Releases page showing tags v0.1.0 and v1.0.0]

Figure C6. Tagged releases